Distribution Platform v2 — Requirements Specification

Oct 8, 2026 · @Claude

1. Purpose and scope

This document defines the target system that replaces the current single-level dispatch prototype: a demand-driven distribution platform for packaged goods, run on a daily ordering and delivery cycle, with dairy in Chennai as the first product profile.

What the system does, end to end

1. Forecasts tomorrow's demand per booth and product with an ML model.
2. Sends each booth a suggested order that auto-confirms unless the booth changes it before the cut-off.
3. Rolls booth orders up into each hub's order to the plant.
4. Allocates stock fairly when supply is short, at both levels.
5. Plans trucks from plant to hubs and vans from hubs to booths.
6. Runs deliveries live: tracking, disruptions, urgent requests, proof of delivery, crates.
7. Closes the day and stores forecast vs actual for every date.

In scope: outbound distribution of packed products, plant → hubs → booths, with web dashboards for managers and three mobile apps (booth ordering, hub ordering, driver).

Out of scope for this version: raw milk collection, plant production, doorstep delivery, payments (see section 22).

Product name. The name comes from configuration (`APP_NAME`). Working proposal: Allocra. Dairy is the first profile; amoxicillin is the second profile planned on the same engine.

How to read this document. Sections 3–6 define the world (network, products, vehicles, people). Sections 7–13 define the daily workflow. Sections 14–16 define screens, time and ML. Sections 17–20 are for implementation. Sections 21–24 cover the demo, limits and build plan. Words like *must* are requirements; *should* are strong defaults; *may* are optional.

2. Current system and what changes

The current prototype plans one level (Madhavaram plant → Chennai "hubs" with litre-based trucks), runs one simulated morning in memory, and handles demand spikes and disruptions well. The target keeps that engine and live behaviour, and rebuilds the data model, time model, roles and apps around it.

Area	Current prototype	Target system	Action
Network	One level: plant → hubs	Two levels: plant → hubs, hub → its booths	Rebuild
Demand unit	Litres per hub	Units per SKU per booth, planned in crates; litres/kg as summary	Rebuild
Vehicles	One fleet, capacity in litres, fixed in code/CSV	Plant trucks + hub vans, capacity in crates and kg, temperature lanes, edited in a UI	Rebuild
Demand source	Typed by manager	ML suggestion → booth app → auto-confirm → hub roll-up	New
Time	One morning, in memory, clock from 03:00	Calendar of dates, daily event timeline, global clock, stored in a database	New
Storage	In memory; lost on restart	SQLite database with migrations	New
Users	One manager screen + driver view	Admin, plant manager, hub manager (web); booth, hub, driver (apps); logins	New
Planner	Greedy heuristic, litres	Same heuristic adapted to crates and lanes, behind a swappable interface (OR-Tools later)	Adapt
Live operations	Reroute, spike handling, rain/traffic, roadblocks, alerts	Same rules, at both levels, with crates and urgent booth requests	Keep + adapt

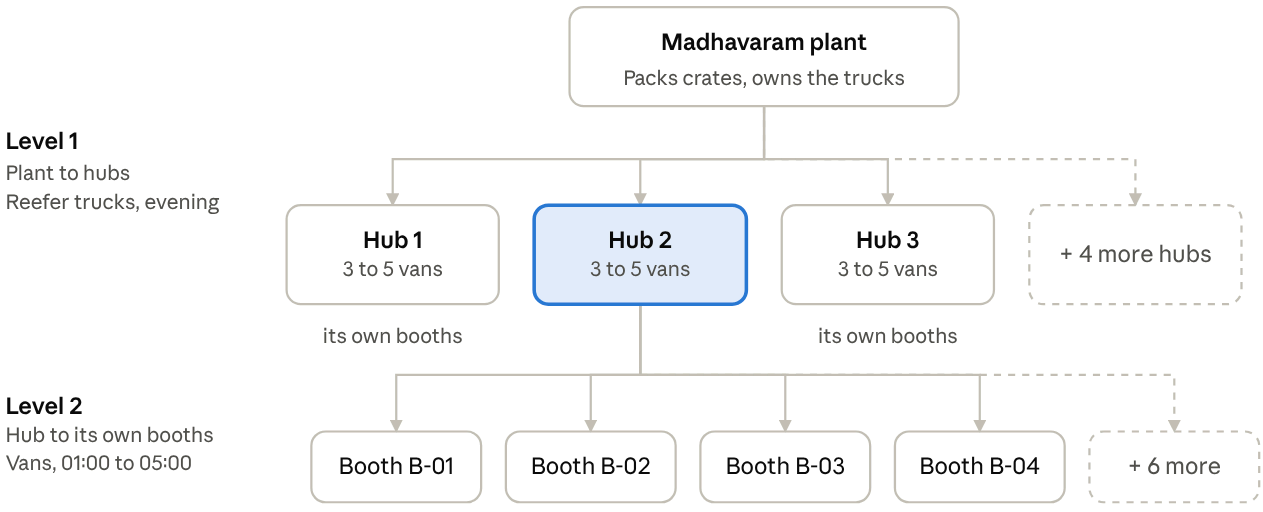
Area	Current prototype	Target system	Action
Routing	OSRM with disk cache, road snapping	Same	Keep
Map, icons, design system	Dark control-room theme, milk icons	Same, extended to two views and three apps	Keep + extend
Driver view	Navigation-style view of one truck	Full driver app: trips at both levels, proof of delivery, crates, issues, messages	Extend
Scenarios	5 scripted mornings	Default network + story mode of a full day	Replace

Removed: litre-based hub demand typed by hand, the fixed default fleet, the single-morning scenario player, and any state kept only in memory.

3. Network model

The network is a tree with two levels: one plant supplies its hubs, and each hub supplies only its own booths.

One plant supplies the hubs; each hub supplies only its own booths



network · 1 plant, 7 hubs, 8–12 booths per hub

Plant trucks carry crates to the hubs in the evening; each hub's vans carry crates to that hub's booths overnight.

Nodes

- **Plant:** the single source. Packs products into crates and holds plant stock per SKU. Default: Madhavaram, Chennai.
- **Hub:** a depot that receives crates from the plant and serves its booths. Has its own vans, a hub manager (web) and a hub ordering user (app). Default: 7 hubs across Chennai.
- **Booth:** a retail point (booth, parlour, shop, distributor) that orders from its hub and receives one delivery per cycle. Default: 8–12 per hub.

Rules (must)

- Every booth belongs to exactly one hub; the assignment is set in the network editor or CSV.
- A hub's vans only serve that hub's booths.
- Exception: a hub manager may lend a booth to a neighbouring hub for one day; the other hub manager accepts; both plans update; it is logged.
- Hub → booth dispatch cannot start until that hub has received its plant delivery (or the manager overrides with stock on hand).

Why a tree and not many-to-many: each booth has one supplier and one route in real distribution, which keeps planning fast and explainable. Several plants serving one city, and automatic booth-to-hub clustering, are roadmap items (section 22).

4. Products and packaging

Everything after the plant moves as packed products in crates or cartons; no loose liquid is carried at any level. Orders are placed in units, planned and loaded in crates, and reported in litres or kg.

Product (SKU) fields: id, name, category, pack size (ml or g), unit type (pouch, cup, carton, tin), units per crate or carton, weight per full crate (kg), temperature lane, shelf life (hours or days), active flag.

Temperature lanes:

- **Chilled (2–8 °C):** milk pouches, curd, buttermilk, paneer, butter.
- **Ambient:** ghee, UHT milk, milk powder, sweets.
- **Frozen (−18 °C or lower):** ice cream. Optional in this version.

A product may only be loaded on a vehicle that supports its lane.

Demo default catalogue (values are assumptions to confirm with a dairy, all editable):

SKU	Pack	Unit	Units per crate/carton	Lane
Toned milk	500 ml	Pouch	24	Chilled
Standardised milk	500 ml	Pouch	24	Chilled
Full cream milk	500 ml	Pouch	24	Chilled
Toned milk	1 L	Pouch	12	Chilled
Curd	500 g	Pouch	20	Chilled
Buttermilk	200 ml	Pouch	40	Chilled
Paneer	200 g	Pack	25 per carton	Chilled
Ghee	1 L	Tin	12 per carton	Ambient

How quantities are shown to users (the same everywhere):

- Booth app: units first, crates second, e.g. "240 pouches · 10 crates".
- Hub app and web views: crates first, units and litres/kg as summary, e.g. "420 crates · 5,040 L".
- Vehicle load: crates used / capacity and kg used / capacity, as bars.
- Partial crates are allowed for booth orders; loading counts a partial crate as one crate.
- Litres = units × pack size; kg = crates × weight per crate.

5. Vehicles and drivers

Each level has its own fleet: refrigerated trucks based at the plant serve hubs, and insulated vans based at each hub serve that hub's booths. Every vehicle is created and edited in the Fleet screen; CSV import is optional.

Field	Meaning	Example
id, name	Unique code and label	VAN-TNG-02
level	plant or hub	hub
home	Plant id or hub id where it starts and ends	Hub T. Nagar

Field	Meaning	Example
type	Reefer truck, insulated van, small reefer van, closed van	Insulated van
capacity_crates	Maximum crates	150
capacity_kg	Maximum payload	1,500 kg
lanes	Lanes it can carry	Chilled, Ambient
refrigerated	Has its own cooling unit	No
speed_kmph	Planning speed in the city	30
cost_per_km, fuel_km_per_l	For cost and fuel reports	₹18 / km, 5 km/L
shift_start, shift_end	When it may work	00:30–06:30
driver	Assigned driver (may change daily)	R. Kumar
status	Active, maintenance, inactive	Active

Drivers: id, name, phone, licence number, home (plant or hub), login PIN, active. A driver is assigned to one vehicle per trip; the assignment can be changed by the manager before dispatch.

Demo default fleet: 3 reefer trucks at the plant (about 400 crates each) and 3–5 insulated vans per hub (100–150 crates each). All values editable.

Fleet screen requirements:

- Table filtered by level and home; add, edit, duplicate, deactivate, delete (delete blocked if the vehicle has trips that day).
- Form with validation (capacity > 0, at least one lane, shift end after start).
- Changes after dispatch apply from the next day; the screen says so.
- Each vehicle has one colour used on every map, list and app.

6. Roles, apps and access

Three roles work in the web app, and three roles use phone apps. The booth and hub apps are for ordering only; all dispatch and live operations happen in the web app.

Role	Where	Main jobs	Sees
Admin / head office	Web	Network, fleet, products, users, settings, calendar, ML accuracy, prototype controls	Everything (read-only for operations)
Plant manager	Web	Review hub orders, approve big hub changes, allocate plant stock across hubs, plan and track plant trucks	All hubs; any hub view read-only
Hub manager	Web	Approve booth changes, review hub order to plant, plan vans, run live operations, urgent requests, lend a booth, messages, day close	Own hub and its booths
Booth	Booth app	Accept or change tomorrow's order, mark closed, track delivery, confirm receipt, urgent request	Own booth only
Hub (ordering)	Hub app	Accept or change the hub's order to the plant, track incoming truck, confirm receipt	Own hub only
Driver	Driver app	Run assigned trips at either level, proof of delivery, crates, returns, issues, messages	Own trips only

Access rules (must):

- Login by phone number + PIN; the server issues a token with role and scope (hub id or booth id).
- Every API request checks role and scope; a booth cannot read another booth's data, a hub manager cannot act on another hub.
- Every action that changes an order, a plan or a trip is logged with user, time and reason.
- The hub app user and the hub manager may be the same person; they still log in to different surfaces.

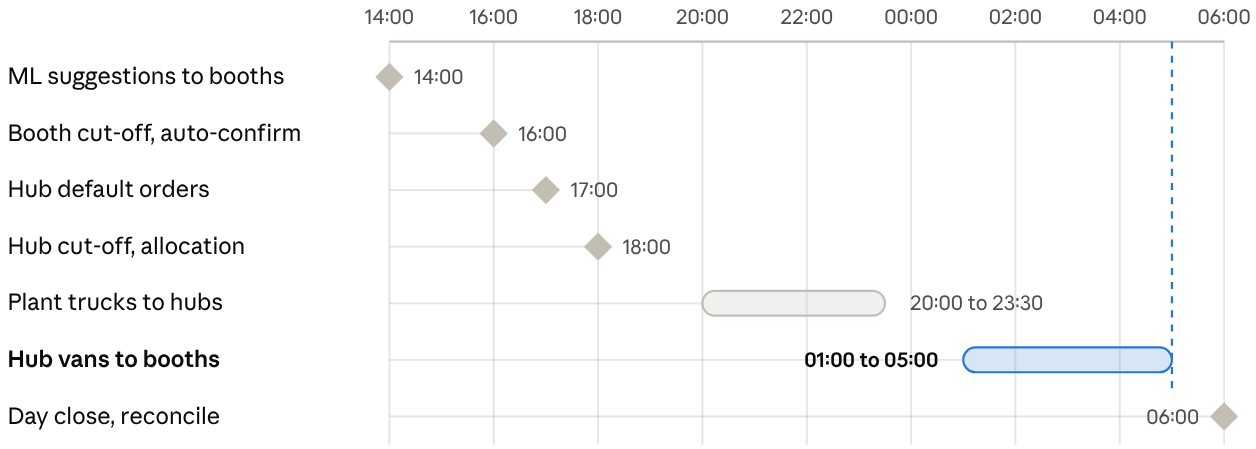
Demo login: a login screen lists seeded users per role for one-tap login, marked "Prototype".

7. Daily cycle

Every delivery day D is prepared the afternoon before: orders lock by 18:00 on D-1, plant trucks run in the evening, and hub vans deliver overnight so every booth is served by 05:00 on D.

Orders lock by 18:00; every booth is served by 05:00

Default daily cycle, from the day before (D-1) to the delivery day (D); all times are settings



daily cycle · D-1 14:00 to D 06:00

Each event runs exactly once per day, in this order; the prototype clock can jump between them (section 15).

What each event does

1. **ML suggestions:** forecasts written for D; each booth gets its suggested order and reason (section 8).
2. **Booth cut-off:** untouched orders auto-confirm; all booth orders lock; pending approvals wait for the hub manager.
3. **Hub default orders:** each hub's order to the plant is computed from its booths' final orders (section 19) and shown in the hub app.
4. **Hub cut-off:** hub orders lock; the plant allocates stock across hubs if short (section 9); each hub then allocates its expected receipt across booths.
5. **Plant trucks to hubs:** trucks are planned and dispatched; hubs confirm receipt in the hub app.
6. **Hub vans to booths:** vans are planned per hub, loaded and dispatched; live operations run until every stop is done (sections 10–12).

7. **Day close:** vans return with empties and returns; reconciliation is stored as the record for D; actual demand is recorded for ML.

Dependencies (must): each event waits for the one before it; a hub's vans wait for its plant delivery; late approvals revert to the default at the next cut-off.

8. Ordering

Orders use an auto-confirmed default: the system proposes a quantity, silence means yes, and only a change needs action before the cut-off.

8.1 Booth order (booth → hub, in the booth app)

1. At suggestion time the booth receives tomorrow's suggested order per SKU, with a one-line reason (e.g. "usual Friday · Deepavali in 2 days · rain expected").
2. Until the booth cut-off the booth may: leave it (it auto-confirms), change any quantity with + / − or by typing, or mark **Closed tomorrow** (order becomes zero).
3. A change beyond ±30 % of the suggestion for a SKU needs hub manager approval and a reason; smaller changes need no approval and the reason is optional.
4. 30 minutes before the cut-off the app reminds the booth: "Your order auto-confirms at 16:00."
5. At the cut-off the order locks. Pending approvals not decided by the hub cut-off revert to the suggestion.

Booth order states: Suggested → Changed / Pending approval → Confirmed (auto or manual) → Locked → Allocated (allocated vs ordered, with reason if short) → Out for delivery (van ETA) → Delivered (booth confirms crates, empties, damage) → Closed.

8.2 Hub order (hub → plant, in the hub app)

1. After the booth cut-off the system computes the hub's default order per SKU (formula in section 19) and shows its breakdown: booth orders, buffer, usable stock at the hub.
2. Until the hub cut-off the hub may leave it (auto-confirms) or change it. A change beyond ±20 % needs plant manager approval and a reason.
3. At the hub cut-off the order locks and goes to plant allocation.

Hub order states: Computed → Changed / Pending approval → Confirmed → Locked → Allocated → In transit (truck ETA) → Received (hub confirms crates and shortages) → Closed.

8.3 After the cut-off

When	Allowed	Handled by
Before cut-off	Any edit by the booth or hub	App, no approval within limits
After cut-off, before dispatch	Change only through a manager	Hub manager (booths) or plant manager (hubs); re-plan
After dispatch	Urgent request for extra units, or refusal of crates at delivery	Control tower: spare crates on vans or a top-up trip (section 11)

8.4 Records

For every order line the system stores: suggestion, p10/p50/p90 forecast, final quantity, who changed it, when, reason, approval result, allocated quantity, delivered quantity. These records feed accuracy reports and the next forecasts.

9. Shortage allocation

When supply of a SKU is less than total orders, the same rule runs at both levels: the plant shares its stock across hubs at the hub cut-off, and each hub shares its expected receipt across booths. Nobody finds out at the counter: every receiver sees allocated vs ordered and the reason before dispatch.

Rule, per SKU (configurable):

1. Remove closed receivers.
2. **Guaranteed share:** each receiver gets min(order, minimum share × order), default minimum share 80 %, served in priority order Critical → High → Normal. If even guaranteed shares don't fit, Critical receivers are filled first, then the rest in proportion to their orders.
3. **Remainder:** what is left is split in proportion to each receiver's unmet order.
4. Round to whole units, then to crates for loading; rounding leftovers go to the receiver with the largest unmet share.

Settings: minimum share %, priority per hub and per booth (with a reason, e.g. hospital nearby), optional channel rule (e.g. booths before bulk or online orders).

Outputs: allocation table per SKU (ordered, allocated, short, %), a notification to each affected booth or hub, and a decision log entry with the rule applied.

Manual override: the plant manager (hubs) or hub manager (booths) may adjust an allocation before dispatch; totals cannot exceed supply, and the override is logged with a reason.

10. Planning and dispatch

One planning engine serves both levels; only the source, receivers and vehicles change. The engine sits behind a Planner interface so the current heuristic can be replaced by OR-Tools later without touching anything else.

Planner input: source (plant or hub), receivers with allocated crates per SKU and delivery windows, vehicles available at that source, products (lanes, weights), road distances and times, dispatch start time.

Planner output: trips; each trip = vehicle, driver, ordered stops, crates per SKU per stop, ETA and slack per stop, km, fuel, cost, utilisation.

Hard constraints (must never be broken):

- Crates and kg on a vehicle within capacity.
- Each product only on a vehicle supporting its lane; a receiver needing two lanes may get two vehicles.
- Every allocated crate on exactly one trip.
- Trips start after dispatch time and end within the vehicle shift.
- A van's crates must be available at its hub (received from the plant, or overridden by the manager using stock on hand).

Objective (in strict order): serve all allocated crates → meet delivery windows → fewest vehicles → least km.

Loading sheet per trip: crates per SKU per stop, loaded in reverse stop order (last stop in first); printable and shown in the driver app.

Level differences:

	Plant → hubs	Hub → booths
Runs at	Plant dispatch event (evening)	Hub dispatch event (night)
Stops per trip	1–3	10–40

	Plant → hubs	Hub → booths
Vehicles	Reefer trucks at the plant	Vans at the hub
Window	Arrive at hub before its van dispatch	Booth ready time (default 05:00)
Live mode	Tracking, delay alerts, disruption reroute	Full live operations (section 11)
Dependency	None	Waits for its plant truck to be received

Re-planning: a manager can re-plan before dispatch after an approval or override; after dispatch only the control tower changes trips (section 11).

Booth lending: a hub manager may move a booth to a neighbouring hub for one day; the other hub manager must accept; both plans update.

11. Live operations

The existing control-tower and disruption logic is kept and applied to both levels, now counting crates instead of litres. Nothing waits on a manager: trucks reroute automatically when any detour exists, and managers are alerted when a decision needs them.

11.1 Disruptions

Type	Placed as	Effect
Rain zone	Circle, 1–8 km	Speed × 0.6 inside
Traffic jam	About 1 km of a road	Speed × 0.4 on that stretch
Roadblock	Point, 150 m radius	Road impassable
Area closure	Circle up to 2 km	All roads inside impassable

- Rain and traffic change speeds; ETAs and risk update live. If a stop turns High or Late, the stops on that trip may be re-ordered (auto or approval, per setting).
- Roadblocks: a vehicle learns of a block 5 sim-minutes after it is placed, or within 300 m of it. Detour found → reroute at once and inform the manager. Detour costly (> 15 min, > 30 % km, or a window broken) → reroute and show an alert with options (keep, hand

stop to another vehicle, skip and notify). No detour → vehicle stops (Blocked) and the manager gets a red alert: call driver, return to base, reassign stop, wait for clearance.

- Clearing a block resumes blocked vehicles at once.
- Network calls never block the clock or the API (budget 4 s per detour search; results cached).

11.2 Urgent requests after dispatch

1. A booth sends an urgent request for extra units of a SKU (booth app).
2. The control tower evaluates: (A) a van already out with spare crates of that SKU inserts the booth after its current stop; (B) a van at the hub makes a top-up trip; (C) both.
3. Choice: most units covered → no window newly broken → least extra km. Auto-applied or sent to the hub manager for approval (setting).
4. The booth sees accepted / partly / rejected, with ETA or reason.

Spare crates: each van may carry a buffer of spare crates per SKU (section 19) for these requests; unused spares return to the hub.

11.3 Decreases and refusals

A booth may refuse crates at delivery (e.g. damaged, over-supplied); the driver records it, the crates stay on the van and return to the hub, and the delivered quantity is updated.

11.4 Decision log

Every automatic or manual decision is logged with time, trip, options compared, numbers and reason, and is visible in the web views.

12. Driver app, deliveries and crates

The driver app runs trips at either level and is where every delivery is proven: crates handed over, empties collected, damage recorded, receiver confirmed.

12.1 Driver app screens

1. **Login** (phone + PIN) → **Today**: assigned trip(s), vehicle, start time, number of stops, total crates.
2. **Loading check:** loading sheet by stop; driver ticks crates loaded per SKU; mismatch flagged to the manager before departure.

3. **Drive:** navigation-style map (heading-up, route, next stop, ETA, slack); a **Navigate** button opens Google Maps to the next stop.
4. **At stop:** crates to deliver per SKU (editable if short or refused), empties collected, returns/leaks/damage with quantity and optional photo. Receiver confirms in their app, or the driver enters the 4-digit code shown in the receiver's app.
5. **Issues:** road closed, breakdown, booth closed, receiver absent, other (sends an alert with location).
6. **Messages and calls:** trip thread with the manager; instructions with Acknowledge; one-tap call buttons.
7. **Trip end:** back at base, empties and returns handed in, summary.

Must also: keep the current trip available offline and sync actions later; show reroutes and new stops with a clear banner; work on low-end Android phones.

Position source: simulator in prototype mode; phone GPS when "Use device GPS" is on (background location must be tested on real phones).

Packaging: installable web app (PWA); wrapped with Capacitor into an Android APK for the demo.

12.2 Proof of delivery

A stop is complete when delivered crates are recorded and the receiver confirms (app tap or code). Unconfirmed stops are flagged "unconfirmed" to the manager. Delivered quantities, not planned ones, become the day's actuals.

12.3 Crate ledger

Crates are returnable assets and are tracked per booth, hub and trip.

- Each delivery records crates out and empties back.
- Balance per booth and per hub = crates received − empties returned, carried day to day.
- Booths and managers see balances; balances above a threshold are highlighted.

12.4 Returns and damage

Leaked, damaged or expired units are recorded at the stop with reason; they reduce delivered quantity and appear in day-close reconciliation.

13. Communication and notifications

All communication happens inside the system, is tied to a trip, booth or hub, and is logged; phone calls are one tap away but are recorded only as "call started".

Message threads:

Thread	Between
Trip	Driver ↔ hub manager (vans) or plant manager (trucks)
Booth	Booth ↔ hub manager
Hub	Hub ↔ plant manager
Broadcast	Manager → all drivers / all booths of a hub (read receipts)

Important messages (instructions, reroutes, cancellations) require **Acknowledge**; unacknowledged ones are flagged to the sender after 5 minutes.

Notifications (in-app centre in every app and web view; browser/push where allowed):

Event	Booth	Hub app	Driver	Hub manager	Plant manager
Suggestion published	Yes				
Cut-off in 30 min	Yes	Yes			
Approval needed / result	Result	Result		Request	Request
Allocation short	Yes	Yes		Yes	Yes
Dispatched / ETA	Yes	Yes	Trip assigned	Yes	Yes
Reroute / delay / blocked	ETA change	ETA change	Yes	Yes	Trucks
Urgent request	Result		New stop	Request	
Delivered / received	Confirm	Confirm		Yes	Yes
Message	Yes	Yes	Yes	Yes	Yes

Rules: short, plain text with the number that matters ("Allocated 36 of 40 crates of Toned 500 ml: supply short"); no more than one reminder per cut-off; English required, Tamil optional.

14. Web views per role

The web app has two operational views built on the same layout (plant → hubs and hub → booths) plus admin screens. Every screen answers three questions at a glance: what is happening, what needs my action, and why.

Common layout: top bar (name, role, date, clock and prototype controls, notifications) · left sidebar (tabs for the role's work) · map in the centre · right sidebar (fleet, decisions, alerts, messages) · timeline strip of today's events under the top bar. No panel floats over the map except toasts and modals.

14.1 Plant manager view (plant → hubs)

- Map: plant and all hubs; hub pins show ordered / allocated / sent / received; plant trucks with routes.
- Hub orders table: default, final, change %, approval actions, breakdown on click.
- Supply panel: plant stock per SKU vs total hub orders; shortage allocation result and override.
- Trucks: trips, load bars, ETA per hub, alerts.
- Click a hub → that hub's view, read-only.

14.2 Hub manager view (hub → booths): the main operations screen

- Map: the hub's area; booth pins coloured by state (not ordered, confirmed, changed, short, out for delivery, delivered, problem); vans with routes in their colours.
- Orders tab: booth orders with suggestion vs final, approval queue, closed booths.
- Supply tab: hub order to plant, incoming truck ETA, received stock, allocation to booths.
- Plan tab: van trips, loading sheets (print), re-plan before dispatch.
- Live tab: alerts, urgent requests, disruptions, decision log, messages.
- Actions: approve, override allocation, lend booth, send instruction, add disruption (prototype), close day.

14.3 Admin view

- Network: map editor for plant, hubs, booths (click to add, drag to move, panel to edit, booth→hub assignment, CSV import/export).
- Fleet, Drivers, Products, Users: tables and forms.

- Settings: cut-off times, limits, buffers, allocation rules, notification rules.
- Both levels on one map with a level toggle; calendar; ML and forecast page; story mode.

14.4 Day close (hub manager and plant manager)

A reconciliation screen per hub and for the plant: ordered vs allocated vs delivered per SKU, shortages, refused and damaged units, crate balances, km, fuel, cost, on-time %. Closing the day stores this as the record for that date.

15. Calendar, global clock and prototype controls

The real system runs on real time; the prototype must show a full day in minutes, any date of the month, and still behave exactly as the real system would. One server-side clock drives every app and screen.

15.1 Calendar

Every date has a record. A month view is available to every web role.

Date state	Shown
Past (history)	Forecast vs actual, ordered vs allocated vs delivered, shortages, on-time %, km, cost, model accuracy
Today (live)	The live dashboard at the current clock time
Future (forecast)	Forecast per booth and SKU, draft hub orders and draft plan; no actuals

Past dates are filled with generated history (section 16) so the calendar is never empty.

15.2 Global clock (prototype)

- Stored on the server (date + time) and broadcast to every web view and app; no app keeps its own time.
- Modes: paused; playing at 1×, 5×, 15×, 30×, 60× or 120×; fast-forward.
- **Set date/time:** moving forward fast-forwards through every event in between; moving backward asks for confirmation and restores that day from its stored record.
- **Next event:** fast-forwards to and runs the next scheduled event (never skips one).

- **Run whole day:** fast-forwards to day close, with live trips at 120×.
- Each event runs exactly once per day and is idempotent; restarting the server mid-day resumes correctly.

15.3 Prototype controls panel

Visible in every web view, clearly labelled "Prototype controls": date picker, time, play/pause, speed, Next event, Run whole day, Reset day, Load default network, and a link to Story mode (section 21). Apps show the clock but not the controls.

Must: running a day by Next event and by Run whole day produces identical records.

16. ML forecasting and synthetic data

No real booth-level sales data is available, so the system generates realistic synthetic history from public patterns, trains on it, and labels every chart and number that depends on it "Synthetic data". The method is real; the accuracy figures describe the method, not real booths.

16.1 Synthetic data

- **Public patterns to research and cite** (in `docs/DATA_SOURCES.md`): weekday variation in packaged milk sales; Tamil Nadu festival and holiday dates for last and current year (Pongal, Tamil New Year, Vinayagar Chathurthi, Deepavali and others); Chennai monthly rainfall and temperature normals (IMD or similar).
- **Generator, per booth × SKU × day, 365 days:** base level by booth type (residential, commercial, near market) × product mix × weekday factor × festival effect (pre-festival rise, festival-day pattern per SKU, e.g. curd and ghee up) × rain effect × temperature effect (buttermilk up in heat) × slow trend × random noise.
- All parameters in one config file; fixed random seed; sample plots saved for a sanity check.
- When a simulated day closes, its actual demand comes from the same generator, so forecast vs actual is always available.

16.2 Model

- One global gradient-boosting model (LightGBM) across all booths and SKUs.

- Features: booth id, SKU, booth type, day of week, day of month, holiday and festival flags, days to next festival, rainfall, temperature, lags (1, 7, 14 days), rolling means (7, 28 days).
- Outputs: p10, p50, p90 quantiles per booth × SKU × date.
- Validation: train on earlier days, hold out the last 60 days.
- Baseline: same weekday last week.
- Metrics: WAPE and MAPE overall, per SKU, per booth type; the model must beat the baseline on the held-out period.

16.3 How forecasts are used

- **Suggestion** for each booth = p50 rounded up to whole units, shown with crates.
- **Reason line** built from the strongest contributing features (e.g. "usual Friday · Deepavali in 2 days").
- **Buffers** (hub order and van spares) from forecast spread p90 − p50 (section 19).
- **Plant pack plan:** total forecast per SKU for tomorrow, shown to the plant manager before hub orders arrive.
- **Learning signal:** suggestion vs final order vs delivered, per booth, stored daily.

16.4 ML page (web)

Accuracy vs baseline, forecast vs actual chart for any booth, SKU and date range, festival effects, and how often booths accept the suggestion unchanged — all labelled "Synthetic data".

17. Data model

All state lives in a database (SQLite for the prototype, PostgreSQL-ready) with migrations; nothing important is kept only in memory.

Entity	Key fields	Relations
Plant	id, name, lat/lon, road-snapped lat/lon, stock per SKU per day	Has hubs, trucks
Hub	id, name, location (raw + snapped), priority, stock per SKU, operating hours, contact	Belongs to plant; has booths, vans

Entity	Key fields	Relations
Booth	id, name, location (raw + snapped), type, priority, ready-by time, vehicle restriction, contact	Belongs to one hub
Product	id, name, pack size, unit type, units per crate, kg per crate, lane, shelf life	Used by order lines
Vehicle	id, level, home, type, capacity crates/kg, lanes, speed, cost, shift, colour, status	Has trips
Driver	id, name, phone, licence, home, PIN	Assigned to trips
User	id, phone, PIN hash, role, scope (hub or booth id)	Acts on everything
Day	date, state (history, live, forecast, closed), events run	Groups all daily records
Forecast	booth, SKU, date, p10, p50, p90, reason	Feeds suggestions
Order	id, date, level (booth or hub), from, to, state, cut-off	Has order lines
Order line	SKU, suggested, final, changed by, reason, approval, allocated, delivered	Belongs to order
Approval	order line, requested by, decided by, result, time	
Allocation	date, level, SKU, receiver, ordered, allocated, rule, override	
Plan / Trip	date, level, vehicle, driver, state, km, cost	Has stops
Stop	trip, receiver, sequence, ETA, slack, window, state	Has delivery lines
Delivery line	stop, SKU, crates planned, delivered, refused, damaged	
Crate ledger	date, receiver, crates out, empties back, balance	
Disruption	type, geometry, created, cleared	Affects trips

Entity	Key fields	Relations
Decision	time, trip or system, type, options, chosen, reason	
Message	thread (trip, booth, hub, broadcast), sender, text, ack	
Notification	user, type, text, read	
Actual demand	booth, SKU, date, units (synthetic)	ML training
Setting	key, value	Cut-offs, limits, buffers, rules
Sim clock	date, time, mode, speed	One row

18. Configuration and data import

Every input can be set up three ways: load the default scenario, add or edit by hand in the UI, or import a CSV.

18.1 Default scenario ("Chennai demo")

- Madhavaram plant.
- 7 hubs spread across Chennai, coordinates verified and snapped to roads.
- 8–12 booths per hub (about 70 total) placed on real roads near each hub, named "<Area> B-01".
- Product catalogue from section 4; fleet from section 5; one driver per vehicle; demo users for every role.
- Loadable from Admin at any time (replaces current data after confirmation).

18.2 Adding on the map

- Click the map to add a hub or booth; the point snaps to the nearest road.
- Rejected with "No road access here" if the snap distance exceeds 300 m (water, marsh, airport).
- Drag to move; the side panel edits all fields; a booth's hub can be changed there.
- Validation: unique name of 3–30 characters, required fields, booth must have a hub.

18.3 CSV import and export

File	Columns (minimum)
hubs.csv	id, name, lat, lon, priority, contact
booths.csv	id, name, lat, lon, hub_id, type, priority, ready_by, vehicle_restriction, contact
products.csv	id, name, pack_size, unit_type, units_per_crate, kg_per_crate, lane, shelf_life
vehicles.csv	id, level, home_id, type, capacity_crates, capacity_kg, lanes, speed_kmph, cost_per_km, fuel_km_per_l, shift_start, shift_end
drivers.csv	id, name, phone, licence, home_id

- Downloadable templates with an example row.
- Row-by-row validation with clear errors ("row 14: no road access at this location").
- Preview before import; mode **Add** or **Replace**.
- Export of current data in the same format.

18.4 Settings (Admin, all editable)

Cut-off and event times, approval limits (booth ±30 %, hub ±20 %), minimum share %, buffer z-values, spare crates rule, booth ready-by default, detection delay and detour thresholds, control mode (auto or approve), notification rules, units display.

19. Formulas and decision rules

One shared module computes every number below; screens and apps display its results and never recompute them. All constants live in Settings.

Units

$$\text{crates} = \left\lceil \frac{\text{units}}{\text{units per crate}} \right\rceil \qquad \text{litres} = \text{units} \times \text{pack size} \qquad \text{kg} = \text{crates} \times \text{kg per crate}$$

Booth suggestion: suggestion = ⌈p50⌉ units per SKU.

Hub default order (per SKU)

```
\text{hub order} = \max\Big(0,\; \sum_{\text{booths}} \text{final order} + z \cdot \sqrt{\sum_{\text{booths}} (p90 - p50)^2} - \text{usable hub stock}\Big)
```

Default z = 1.0; usable stock counts only units within shelf life at dispatch time (chilled stock is usually zero).

Spare crates per van (per SKU): spare = round(z_van × √Σ (p90 − p50)² over the van's booths), default z_van = 0.5, capped by capacity left after planned crates.

Approval limits: booth change |final − suggestion| / suggestion > 30 % → approval; hub change > 20 % → approval.

Shortage allocation: guaranteed min(order, 80 % × order) by priority, remainder in proportion to unmet orders (section 9).

Time

Quantity	Formula
Travel time	road km ÷ (speed × rain 0.6 × traffic 0.4 where they apply)
Unloading time	5 min + 0.2 min per crate (default, editable)
ETA	departure + travel + unloading at earlier stops
Slack	delivery window end − ETA
Risk	slack ⩾ 30 min Low · 10–30 Medium · 0–10 High · < 0 Late

Cost: fuel L = km ÷ fuel_km_per_l; cost = km × cost_per_km; utilisation = crates loaded ÷ capacity. The return trip to base is included.

Planner objective: all allocated crates → windows met → fewest vehicles → least km (section 10).

Urgent request choice: most units covered → no window newly broken → least extra km (section 11).

Roadblock decision: detour found → reroute; costly if > 15 min or > 30 % km or a window broken → reroute + alert; none → stop + alert (section 11).

Forecast accuracy: WAPE = Σ|actual − forecast| ÷ Σ actual; reported against the same-weekday-last-week baseline.

20. Non-functional requirements and technology

Technology (keep what works):

Layer	Choice
Backend	Python, FastAPI, SQLAlchemy/SQLModel, Alembic, SQLite (PostgreSQL-ready)
Real-time	WebSocket push of clock, trips, alerts; auto-reconnect
Routing	OSRM with disk cache and road snapping; OpenRouteService optional
Planner	Current heuristic behind a Planner interface; OR-Tools later
ML	Python, LightGBM, pandas; offline training script + daily inference
Web app	React + Vite, MapLibre, shared design system
Mobile apps	Same React codebase as installable PWAs (booth, hub, driver); driver wrapped with Capacitor into an Android APK
Auth	Phone + PIN, JWT with role and scope

Reliability

- A clock tick finishes in under 200 ms; API p95 under 100 ms at demo scale (1 plant, 7 hubs, 70 booths, ~30 vehicles).
- Network calls never run inside the clock tick or while holding a lock; one failing call never stops the simulation.
- Restarting the server loses nothing and resumes the day.
- Apps keep working through short disconnects and show "Reconnecting…".

Consistency

- The same quantity, ETA or status shows the same value in every view and app.
- One design system; status colours mean the same thing everywhere; each vehicle has one colour everywhere; no overlapping panels; no default browser controls.

Security and audit

- Role and scope checked on every request; PINs stored hashed.
- Every change to an order, allocation, plan or trip logged with user, time, reason.

Usability

- Phone apps: large touch targets, readable in sunlight, usable one-handed, work on low-end Android phones.
- English required; Tamil optional.

Demo deployment: LAN mode — the start script prints the LAN address and a QR code per app so phones on the same Wi-Fi can open them; an online deployment is optional.

21. Demo and story mode

Story mode runs one complete day in about 7 minutes with a narration card at each step, so a presenter can explain the idea and the jury can see every role act on the same data. Each step can be jumped to directly; the clock only pauses when the presenter presses Pause.

Step	What is shown	Screen / app
1. Calendar and ML	Yesterday's forecast vs actual; tomorrow's suggestions and why	Web calendar, ML page
2. Booth ordering	One booth accepts, one changes +40 % (needs approval), one marks closed	Booth app
3. Booth cut-off	Auto-confirm; hub default order computed with its breakdown	Hub manager web, hub app
4. Hub ordering and shortage	Hub adjusts; plant allocates under a deliberate shortage on one SKU	Hub app, plant web
5. Plant → hubs	Trucks dispatched, tracked, received at hubs	Plant web, hub app, driver app

Step	What is shown	Screen / app
6. Hub → booths	Van plan, loading sheets, vans leave	Hub manager web, driver app
7. Live	Roadblock reroute, urgent booth request, proof of delivery	Hub manager web, booth app, driver app
8. Day close	Reconciliation and the stored record in the calendar	Web

Demo setup: laptop for the web views; three phones (or browser windows) logged in as booth, hub and driver; LAN mode with QR codes; route cache pre-warmed so the demo works without a fast connection.

Fallback: if a phone fails, the web view can open any app in a phone frame ("View as booth T. Nagar B-03").

22. Out of scope and roadmap

These are deliberately not built in this version; each can be added later on the same engine and data model.

Item	Why not now	Later
Raw milk collection (tankers from coolers to plant)	Different users (procurement), liquid in litres, quality tests per load	Collection routing module with compartments, seals, quality
Plant production planning	Inside the plant, not distribution	Pack plan from forecasts is already shown
Several plants serving one city (many-to-many)	Harder to solve and explain; not needed for one plant	Multi-source assignment
Automatic booth-to-hub clustering	A setup decision, rarely changed	Network design screen using capacitated assignment
OR-Tools planner	Algorithm upgrade parked	Swap behind the Planner interface; benchmark vs current

Item	Why not now	Later
Doorstep delivery to consumers	Different business model	Same engine with a subscription order cycle
Payments, credit, invoicing	Not needed to prove the workflow	Integrate with existing billing
Native iOS apps	Android covers the target users	PWA already works in iOS browsers
Real GPS hardware, temperature loggers	Hardware	Phone GPS first, IoT later
Second product profile (amoxicillin)	Dairy first	Same engine with pharma rules (batch, expiry, first-expiry-first)

23. Assumptions and open questions

The values below are working assumptions; each is a setting, and each should be checked with someone who works in dairy distribution before it is presented as real.

Assumptions

- City milk in Chennai reaches booths before about 05:00 from an overnight dispatch wave.
- Booths order the day before through an indent with a fixed cut-off.
- Units per crate, crate weight and vehicle capacities in sections 4–5 are typical, not measured.
- Hubs keep little or no chilled stock overnight; usable stock is mostly ambient products.
- Each booth is served by one hub, one delivery per day.
- Demand data is synthetic, built from public patterns (section 16).

Open questions

- [ ] What are the real cut-off and dispatch times used by a Chennai dairy?
- [ ] How many pouches fit in a standard crate for each pack size?
- [ ] Is there an evening delivery cycle that should be modelled as a second cycle?
- [ ] Should booths be able to order the full product range, or only milk and curd at night?
- [ ] What minimum share % is acceptable to booths when supply is short?

- [ ] Do hub vans belong to the dairy or to contractors paid per trip or per day (affects "fewest vehicles" savings)?
- [ ] Final product name (working proposal: Allocra).

24. Build phases and acceptance criteria

The migration is built in eight phases, in order; each must pass its acceptance checks before the next starts, because later phases depend on the data model and the clock.

Phase	Delivers	Accepted when
0. Spec	This document turned into the project's CLAUDE.md	Every later prompt refers to it; no conflicting constants
1. Data and setup	Database, data model, default scenario, map editor, fleet and product screens, CSV import/export	Restart loses nothing; every booth has exactly one hub; bad CSV rows show clear errors; points in water are rejected
2. Time	Global clock, daily event timeline, calendar, prototype controls	A day run by Next event equals a day run by Run whole day; jumping across events runs each once, in order
3. ML	Synthetic history, model, forecasts, reasons, ML page	Deterministic with a seed; beats the baseline on held-out days; forecasts exist for every booth × SKU for today + 7 days
4. Ordering	Logins and roles, booth app, hub app, approvals, cut-offs, notifications	Untouched orders auto-confirm; ±30 % / ±20 % rules enforced; closed booth gets nothing; locked after cut-off; a booth can't see another booth's data
5. Planning	Shortage allocation, planner with crates and lanes, loading sheets, plant and hub web views	No capacity or lane violation; every allocated crate on exactly one trip; hub dispatch waits for receipt
6. Live	Live trips at both levels, driver app (PWA + APK), proof of delivery, crate ledger, messaging, day close	Full day at 120× with booth-confirmed deliveries; crate ledger balances; offline driver actions sync

Phase	Delivers	Accepted when
7. Demo	Story mode, LAN mode, consistency pass, end-to-end tests, docs	Story mode runs start to finish without errors; same numbers in every view; no overlapping panels at desktop and phone sizes

Definition of done for the whole system: a presenter can run one full day in story mode, show every role acting on the same live data, and every number shown can be traced to a stored record and a formula in section 19.